

Metasense: Research Engineer Technical Screening Challenge

Malith Jayalath

August 15, 2026

Question 1

1 System Overview and Assumptions

To address the challenge of continuously logging photoplethysmography (PPG) and electrocardiogram (ECG) waveforms without using DMA, I propose an interrupt-driven Quad-SPI NOR flash storage system with double-buffered RAM. The aim is to separate time-critical data acquisition from the relatively slow flash programming operations.

1.1 Assumptions

- **MCU:** STM32F412VG, ARM Cortex-M4
- **MCU clock speed:** 100 MHz
- **Storage interface:** Quad-SPI, Bank 1 with Quad SPI lines
- **DMA:** Not used
- **Transfer method:** Interrupt-driven QSPI
- **Flash device:** Winbond W25Q64JV 64-Mbit QSPI NOR flash
- **Flash capacity:** 8 MB
- **Page size:** 256 bytes
- **Sector size:** 4 KB
- **Assumed QSPI clock:** 10 MHz
- **Required continuous logging rate:** 10 Mb/min

2 Timing and Buffer-Sizing Arithmetic

2.1 Required Throughput

The required continuous logging rate is approximately

$$R_{\text{acq}} = \frac{10 \times 10^6}{60} \text{ bit/s} = 20833.33 \text{ byte/s} \approx 20.83 \text{ kB/s}. \quad (1)$$

Hence, the storage subsystem must sustain at least approximately 20.83 kB/s continuously.

2.2 QSPI Bus Throughput

Assuming a QSPI clock frequency of 10 MHz and four data lines, the theoretical data throughput is,

$$R_{\text{QSPI}} = 10 \times 10^6 \times 4 \text{ bit/s} = 5 \times 10^6 \text{ byte/s} \approx 5 \text{ MB/s.} \quad (2)$$

This is much higher than the required 20.83 kB/s. Therefore, the QSPI bus speed is not the main bottleneck. The page-program and sector-erase times of the NOR flash are more important.

2.3 Buffering Strategy

A double-buffering implementation using two 8 KB RAM buffers is proposed. During acquisition, incoming data is stored in Buffer A. When Buffer A becomes full, acquisition immediately switches to Buffer B while the storage task writes Buffer A to the QSPI flash.

The time required to fill one 8 KB buffer is

$$T_{\text{fill}} = \frac{8192 \text{ byte}}{20833.33 \text{ byte/s}} \approx 0.393 \text{ s} = 393 \text{ ms.} \quad (3)$$

Thus, after one buffer becomes full, the storage subsystem has approximately 393 ms to write it before the second buffer also becomes full.

The 8 KB buffer size is selected because it is an integer multiple of the 256-byte flash page size and also provides a reasonable margin against temporary flash-programming delays.

2.4 Writing an 8 KB Buffer

The W25Q64JV flash has a page size of 256 bytes. Therefore, the number of pages required for an 8 KB buffer is

$$N_{\text{pages}} = \frac{8192}{256} = 32. \quad (4)$$

For the selected flash device the maximum page-program time is 3 ms, making the worst-case programming time for 32 pages approximately,

$$T_{\text{program}} = 32 \times 3 \text{ ms} = 96 \text{ ms.} \quad (5)$$

The time required to transfer 8 KB over the 40 Mbit/s QSPI data path is approximately

$$T_{\text{bus}} = \frac{8192 \times 8}{40 \times 10^6} = 1.6384 \times 10^{-3} \text{ s} \approx 1.64 \text{ ms.} \quad (6)$$

A total time for writing one 8 KB buffer is therefore,

$$T_{\text{write}} \approx T_{\text{program}} + T_{\text{bus}}. \quad (7)$$

Hence,

$$T_{\text{write}} \approx 96 \text{ ms} + 1.64 \text{ ms} = 97.64 \text{ ms.} \quad (8)$$

Since

$$T_{\text{write}} < T_{\text{fill}}, \quad (9)$$

or numerically,

$$97.64 \text{ ms} < 393 \text{ ms}, \quad (10)$$

the system can complete the flash write before the alternate buffer becomes full. The approximate timing margin is

$$T_{\text{margin}} = 393 \text{ ms} - 97.64 \text{ ms} = 295.36 \text{ ms}. \quad (11)$$

Therefore, the proposed system has approximately 295 ms of timing margin per 8 KB buffer.

3 Non-DMA Operation and CPU Availability

The QSPI peripheral is operated in interrupt-driven mode instead of DMA. The CPU starts a page-program transaction and supplies data through the QSPI FIFO. FIFO-threshold and transfer-complete interrupts are used to service the QSPI peripheral.

After the page data has been transferred, the flash performs the actual programming operation internally. During this period, the CPU can continue handling PPG/ECG acquisition.

The acquisition interrupt is assigned a higher priority than the storage task. Therefore, while Buffer A is being written to flash, acquisition continues into Buffer B.

Stage	Acquisition	Storage
1	Fill Buffer A	Idle
2	Fill Buffer B	Write Buffer A
3	Fill Buffer A	Write Buffer B
4	Fill Buffer B	Write Buffer A

This prevents the CPU from being blocked by flash operations for long periods and keeps the acquisition path responsive.

4 Wear Levelling and Write Amplification

A conventional filesystem is not necessary for a continuous data logger. Instead, the flash can be used as an append-only circular log. Data is written sequentially in 256-byte pages. Writing full pages reduces unnecessary partial-page operations and therefore reduces write amplification.

5 Detection and Recovery of Dropped or Delayed Writes

Each stored data block contains a small header consisting of **payload, increasing sequence number, timestamp, payload length, and CRC checksum**.

For example, valid blocks may have the sequence,

$$1000, 1001, 1002, 1003, 1004.$$

If the stored sequence becomes,

$$1000, 1001, 1003, 1004,$$

the system can detect that block 1002 was lost.

The CRC32 checksum is used to identify corrupted or partially written blocks. A RAM buffer is not marked as free until the corresponding flash programming operation has completed successfully.

If a write is delayed beyond a predefined timeout, the buffer remains valid and the write can be retried. A storage error counter is also incremented.

After a reset or unexpected power loss, the firmware scans the latest records and identifies the most recent block with a valid header, sequence number, and CRC. Any incomplete final block is rejected and logging resumes from the next valid location.

If both RAM buffers become full because storage is delayed for too long, the firmware sets an explicit `storage_overrun` flag and increments a dropped-data counter. Therefore, data loss is detected instead of silently overwriting unwritten data.

6 Overall Throughput

The required throughput is

$$R_{\text{required}} = 20.83 \text{ kB/s.} \quad (12)$$

Using the conservative 97.64 ms write time for an 8 KB buffer, the effective storage throughput is approximately

$$R_{\text{storage}} = \frac{8192 \text{ byte}}{0.09764 \text{ s}} \approx 83868 \text{ byte/s} \approx 83.9 \text{ kB/s.} \quad (13)$$

Hence, when using worst-case flash-programming timing, the proposed subsystem provides a huge margin with the required throughput.

7 Issues and solutions

Flash device sector erase operations can take significantly longer than page programming. Around 300 ms max for the proposed flash device. Therefore, sectors should either be pre-erased before acquisition starts or erased sufficiently far ahead of the active write pointer. This is a considerable factor. Therefore, my proposal is to use a buffer pool instead of a simple ring buffer to address this issue.

8 Conclusion

The proposed system uses an STM32F412VG Cortex-M4 operating at 100 MHz together with a W25Q64JV QSPI NOR flash. The required data rate is approximately 20.83 kB/s, while conservative flash-programming calculations indicate an effective write throughput of approximately 83.9 kB/s.

Two 8 KB RAM buffers allow acquisition and storage to operate concurrently without DMA. Sequential page-aligned writes and circular sector allocation reduce write amplification and distribute flash wear. Sequence numbers, CRC32 checks, write timeouts, and explicit overrun counters ensure that delayed, corrupted, or dropped writes are detected rather than silently losing acquired data.

9 References and sources

- [1] xenForo *SD Datalogging - Best Practice in 2019*. Available at: <https://forum.pjrc.com/index.php?threads/sd-datalogging-best-practice-in-2019.55114/>
- [2] Electronic stack exchange, *How can I write high speed A/D data to an SD card*. Available at: <https://electronics.stackexchange.com/questions/216082/how-can-i-write-high-speed-a-d-data-to-an-sd-card>

- [3] STMicroelectronics community, *How to write data efficiently to an SD card*. Available at: <https://community.st.com/stm32-mcus-products-25/how-to-write-data-efficiently-to-an-sd-card-127829>
- [4] STMicroelectronics community, *Best practice using DMA to read from UART and write to SD?*. Available at: <https://community.st.com/stm32-mcus-products-25/best-practice-using-dma-to-read-from-uart-and-write-to-sd-76190?tid=76190&fid=25>
- [5] The SSD Guy Blog, *What is Write Amplification?*. Available at: <https://thessdguy.com/what-is-write-amplification/>
- [6] EEVblog, *SD Card writing speed*. Available at: <https://www.eevblog.com/forum/microcontrollers/sd-card-writing-speed/>
- [7] EEVblog, *Serial flash (QSPI Nor flash) - expect sustained write throughput (datalogging)?*. Available at: [https://www.eevblog.com/forum/microcontrollers/serial-flash-\(qspi-nor-flash\)-expect-sustained-write-throughput-\(datalogging\)/](https://www.eevblog.com/forum/microcontrollers/serial-flash-(qspi-nor-flash)-expect-sustained-write-throughput-(datalogging)/)
- [8] STMicroelectronics community, *SD card taking time to store data*. Available at: <https://community.st.com/stm32-mcus-embedded-software-32/sd-card-taking-time-to-store-data-23365>
- [9] Winbond Electronics Corporation, *W25Q64JV: 3V 64-Mbit Serial Flash Memory with Dual/Quad SPI*, Datasheet. Available at: <https://www.alldatasheet.com/datasheet-pdf/view/1243794/WINBOND/W25Q64JV.html>
- [10] STMicroelectronics, *STM32F412xE STM32F412xG datasheet*, <https://www.st.com/resource/en/datasheet/stm32f412vg.pdf>